

atomOS_JK

v3.6.1

Blend + otan2

Integration specification

Static log-polar space. Moving pinion.

Moving reference axis. Separate one-bit predicates.

Tom Klootwijk

Author-supplied identifier	NL200678942
Author-supplied date	10-07-1990
Named integration target	atomOS_JK_v3.6.1
Imported specification	ASA v3.1.0 blend integration
New source	otan2 operator math / seven pages
Document revision	I1 / 13 September 2026

Integrated material

The A–W–B–N blend record, ASA/NA/fringe compatibility, the literal otan2 ratio, the axis-relative form, explicit angular conventions and an additive JK-host integration contract. The operator name retains the source's letter o.

Version-targeted addendum, with the new seven-page source preserved in an appendix. The target release number is retained; no unseen v3.6.1 implementation is represented as modified.

Integration record

This document carries the prior **ASA v3.1.0 blend integration** into a formal addendum addressed to **atomOS_JK_v3.6.1**, and joins it to the newly supplied **otan2** source. The integration preserves the former aperture contract while adding a separate description of orientation in the static log-polar chart.

Release item	Disposition
Target name	atomOS_JK_v3.6.1, exactly as supplied by the author.
Document revision	I1. This is not an automatic change to a v3.6.2 or v3.7.0 engine release.
Prior integration inspected	The 20-page ASA v3.1.0 blend PDF, including its original five-page source appendix.
New material inspected	Seven-page <i>ORANGE NATO otan2 operator math</i> PDF.
Target baseline availability	No v3.6.1 master specification, source tree or binary was supplied among the available files.
Delivered change	A source-linked integration specification and merge contract, not a claim of an in-place code or master-document merge.
Default semantics	Append typed blend/otan2 records; do not change the host's state transition, aperture or J/K inputs.

Source and definition keys

P denotes the inspected prior integration PDF. **C** denotes its five-page blend source, which is also separately available. **O** denotes the newly supplied otan2 document. **A** denotes the earlier WHITE KING corpus, used only for its J–K operator context. Page references identify the original PDF order, not a reconstructed text chunk.

I01–I08 identify decisions made in this integration. A source expression is reproduced before any separately labelled completion. The mathematical consequences of a definition are not presented as additional claims already supplied by the source.

The central distinction

The circle-plus blend evaluates encoded bit classes. The new otan2 expression evaluates an orientation from two log-polar increments. Neither output is silently substituted for the other, for a J/K input, or for the inherited active-fringe mask.

Contents

Integration record	2
1 The blend integration carried forward	4
2 The JK host boundary	5
3 The new document, in its own order	6
4 Typed log-polar state and moving-axis fields	7
5 The literal otan2 profile and a separate completion	8
6 Wrapping, angular increments and interval W	9
7 The static chart, moving pinion and moving axis	10
8 Integration into the named v3.6.1 host	11
9 Explicit optional profiles and preserved boundaries	12
10 Status contract and worked examples	13
11 Algebra checks executed for this integration	14
12 Integration register and concrete merge checklist	15
13 Provenance and source appendix	16

Reading route

Sections 1–2 carry the earlier integration forward. Sections 3–7 preserve and type the new source. Sections 8–9 define the host integration and prove its recovery conditions. Sections 10–12 provide examples, finite checks and the merge register. Section 13 identifies sources and the untouched source appendix.

1 The blend integration carried forward

1.1 The retained four-word ASA record

For a fixed logical cell dictionary, the prior integration defines

$$a = x \wedge M_{\text{ASA}} \wedge V, \quad (1)$$

$$n = a \wedge M_{\text{NA}}, \quad (2)$$

$$h = \text{popcount}(n \wedge M_{\partial}), \quad (3)$$

$$y = \begin{cases} 0, & h > 0, \\ n, & h = 0, \end{cases} \quad F_{\text{ASA}}(x; M) = (a, n, h, y). \quad (4)$$

Here x is occupancy, V is the valid-bit mask and M_{∂} is the boundary mask. The order is ASA, then NA, then population count and *whole-word* disposition. This is carried from P, not independently inferred to be the unseen JK v3.6.1 binary's ABI. [P, p. 4]

The inherited optional active-fringe profile replaces M_{ASA} by $M_{\text{ASA}} \wedge M_{\text{fringe}}$. Its neutral mask is $M_{\text{fringe}} = V$. The default profile retains the original core and attaches a companion pinion record. [P, pp. 9–11]

1.2 A–W–B–N and the encoded blend

The source's notation is retained with the established namespaces: A_{axis} is its reference axis; W is a time interval; B_{ϕ} is the named kinematic descriptor; and N_{pinion} is the north-pinion datum. The phrase “double lowercase phi” remains the descriptor's source label rather than being silently interpreted as $\ddot{\phi}$, ϕ^2 or a particular matrix. [P, p. 6][C, p. 1]

The original expression and its explicitly binary reading remain distinct:

$$s_{\text{source}} = N \oplus (A + \phi(W)) \pmod{2}, \quad s_{\text{blend}} = n_b \oplus a_b \oplus \beta_b. \quad (5)$$

The latter requires $n_b, a_b, \beta_b \in \{0, 1\}$ from a declared encoding. Neither a real-valued angle nor an entire axis becomes a bit merely by appearing in this equation. [P, p. 7][C, pp. 1–2]

1.3 The three predicates remain separate

P distinguishes s_{align} , s_{limit} and s_{fringe} . Their respective one-states mean an alignment discrepancy, an envelope/threshold event, and active-fringe acceptance. Missing definitions have an *unspecified* status rather than a guessed zero. The otan2 record does not collapse these three meanings. [P, pp. 8–9]

2 The JK host boundary

2.1 A named release is not a recovered source tree

The user-selected target is atomOS_JK_v3.6.1. Its exact public interfaces, bit layout, release changes and state-update schedule are not included in P or O. The host is therefore represented by an opaque transition

$$E_{361} : (S, I) \mapsto (S^+, R_{361}). \quad (6)$$

No missing release history between ASA v3.1.0 and JK v3.6.1 is invented. The compatibility contract in Section 8 operates on this interface without specifying its internals. [I01]

2.2 The source's J–K role

The earlier corpus names JK as a J–K flip-flop and explicitly describes set and toggle cases. For an abstract binary J–K interpretation, the retained truth-table completion is

$$q^+ = (J \wedge \neg q) \vee (\neg K \wedge q), \quad q, J, K \in \{0, 1\}. \quad (7)$$

This equation describes the *Boolean J–K model* used for this addendum's compatibility checks. It is not a reverse-engineered implementation of v3.6.1. [A, pp. 53–55][I02]

J	K	q^+	Mode
0	0	q	Hold
1	0	1	Set
0	1	0	Reset
1	1	$1 - q$	Toggle

The table leaves the origin of J , K and the update event to the host. Neither C nor O specifies a map from an angular residual to these two inputs. Consequently, otan2 is an annotation by default, not an automatic toggle, reset or set. No test of a mathematical J–K table establishes the absence of implementation errors in a particular release.

2.3 Existing state is not renamed

J_{JK} and K_{JK} must remain distinct from any earlier jitter register named J . The axis field A_{axis} must remain distinct from the ASA aperture mask. N_{pinion} must remain distinct from the NA mask, and the source's B_ϕ descriptor must remain distinct from the boundary word. These distinctions preserve P's namespacing when its record is attached to a JK-host interface. [P, pp. 6, 12]

3 The new document, in its own order

O begins with ordinary `atan2`, proposes two log-polar interpretations under that spelling, and then records the author's correction: "*no literally otan2 not atan2*". The correction is retained. The integration does not silently rename the author's operator. [O, pp. 1–6]

O page	Source content	Integration role
1–2	Cartesian <code>atan2</code> ; quadrant table; argument-order note.	Background retained separately from the custom name.
2–3	Log-polar coordinates; direct phase projection; difference of angular coordinates.	Two preparatory alternatives, not the final literal <code>otan2</code> definition.
4	Code for principal phase normalization.	A wrapped-phase helper with an explicit range convention.
5	Author insists on <code>otan2</code> ; response uses "Orthogonal / Orthographic Tangent" wording.	Retain the custom name and source descriptor without treating the descriptor as a standardized API.
6	$\text{otan2}(\Delta\theta, \Delta\rho) = \arctan(\Delta\theta/\Delta\rho)$; scale and spiral descriptions.	Literal ratio profile, with its domain stated rather than silently replaced.
6–7	Static chart, moving spoke/pinion and a moving "eigenvalue axis."	Typed increments and a separately supplied axis angle in the same frame.
7	Ratio-angle minus the axis angle.	Literal axis-relative expression plus a separately named principal-angle view.

3.1 The page-5 illustrations

Page 5 contains a small Cartesian angle sketch and paired image/log-polar thumbnails in the source's search-result block. They belong to the preceding `atan2`/registration discussion. They do not provide a numerical `otan2` LUT, eigenbasis, memory format or J/K transition rule. The source appendix preserves their position in the document.

3.2 The final operator takes increments

The final ratio concerns $\Delta\rho$ and $\Delta\theta$, not simply the point coordinates (ρ, θ) . Extracting the phase of a point, subtracting two phases, and measuring a direction of change are three different operations. The integration keeps that progression visible rather than treating the first and last expressions as interchangeable. [O, pp. 3, 6–7][I03]

4 Typed log-polar state and moving-axis fields

The carried chart convention is

$$\rho = \ln(r/r_*), \quad r_* > 0, \quad r > 0, \quad \phi \in \mathbb{R}/(2\pi\mathbb{Z}). \quad (8)$$

The fixed reference length r_* follows the prior normalized log-radius convention. O writes $\rho = \ln r$; selecting the same r_* for a compared pair is an explicit integration convention, not a changed spelling of the source equation. [P, p. 4][O, pp. 2–3][I04]

Source / symbol	Typed field	Meaning
ρ	log_radius	Finite real value in the declared chart.
θ / lower-case ϕ	phase	Angular coordinate; source spelling recorded as an alias.
$\Delta\rho$	delta_rho	Difference or integrated increment over the specified interval.
$\Delta\theta$ / $\Delta\phi$	delta_phase	Angular increment with a declared turn convention.
W	time_interval	Common positive interval when increments are converted to rates.
$\phi_{\text{eigenvalue}}$	axis_angle / α	Supplied orientation in the same declared log-chart frame.
A_{axis}	axis_reference	Prior source role; only mapped to α when its representation permits it.
N_{pinion}	pinion_reference	Prior north/datum role, not a ratio denominator.

4.1 What “eigenvalue axis” supplies

O uses the phrase for a preferred local direction and explains it in terms of eigenvectors. It supplies no matrix, spectrum, sign selection or normalization. The integration therefore accepts an axis orientation as input; it does not manufacture an eigenvector from the phrase. [O, p. 7]

An unoriented axis is unchanged by a half-turn. A directed reference is unchanged only by a full turn. The field `axis_semantics` must distinguish these when used numerically. The literal source ratio loses a simultaneous reversal of both increments, which is compatible with a line slope but insufficient to distinguish opposite directed motions.

4.2 Missing is distinct from zero

Each result includes a definition status. Missing turn information, an absent axis or a frame mismatch must not be represented as perfect zero residual. A valid zero is an actual value; an unavailable orientation is an unavailable value. This extends the prior predicate-status convention without reinterpreting its one-bit states. [P, p. 8][I05]

5 The literal otan2 profile and a separate completion

5.1 Literal source definition

Preserve the equation exactly as the source's ratio profile:

$$\text{otan2}_{\text{source}}(\Delta\phi, \Delta\rho) = \arctan\left(\frac{\Delta\phi}{\Delta\rho}\right), \quad \Delta\rho \neq 0. \quad (9)$$

For finite real inputs with a nonzero denominator, its value lies in $(-\pi/2, \pi/2)$. The source's axis-relative form is the unwrapped difference

$$e_{\text{source}} = \text{otan2}_{\text{source}}(\Delta\phi, \Delta\rho) - \alpha. \quad (10)$$

Neither quadrant restoration nor a final wrap is present in O's last displayed formula. Those steps are not silently inserted into Eqs. (9)–(10). [O, pp. 6–7]

When $\Delta\rho = 0$, the written ratio is undefined. When both increments are zero, there is also no direction of change. These cases receive distinct statuses. A pure angular case is not assigned zero merely because the original denominator vanishes. [I05]

5.2 Optional directed completion: a distinct profile

For a caller explicitly selecting `directed_completion_1`, define

$$\text{otan2}_{\text{dir}}(\Delta\phi, \Delta\rho) = \text{wrap}_{\pi}(\text{atan2}(\Delta\phi, \Delta\rho)), \quad (\Delta\rho, \Delta\phi) \neq (0, 0), \quad (11)$$

where wrap_{π} has range $[-\pi, \pi)$. This completion uses the usual two-argument primitive internally while retaining `otan2` as the custom profile name. It is an *explicit addition*, not the literal equation supplied by O. Its axis-relative value is $e_{\text{dir}} = \text{wrap}_{\pi}(\text{otan2}_{\text{dir}} - \alpha)$. [I06] The primitive's argument order and quadrant behavior are documented in Python's standard math reference. [E1]

5.3 Relation between profiles

For $\Delta\rho > 0$, the source and directed orientations agree. For $\Delta\rho < 0$ they differ by a half-turn before principal normalization. The ratio is unchanged by replacing both increments by their negatives, so it cannot alone preserve directed quadrant information. The `profile` field must identify which result is reported.

The source descriptor “orthogonal / orthographic” is preserved as terminology. Its displayed formula contains no additional $\pi/2$ normal rotation. None is imposed by this integration.

6 Wrapping, angular increments and interval W

6.1 Principal projection is not turn recovery

Define the selected principal representative by

$$\text{wrap}_\pi(u) = u - 2\pi \left\lfloor \frac{u + \pi}{2\pi} \right\rfloor \in [-\pi, \pi). \quad (12)$$

In this convention $+\pi$ is represented by $-\pi$. O's page-4 expression has this half-open range, even though nearby prose writes a closed interval and uses the phrase “phase-unwrapping wrapper.” The expression wraps a phase; it does not reconstruct omitted revolutions. [O, pp. 3–4][I04]

The earlier direct-projection and phase-difference discussions are retained as

$$P(\rho, \phi) = \text{wrap}_\pi(\phi), \quad D(\phi_0, \phi_1) = \text{wrap}_\pi(\phi_1 - \phi_0). \quad (13)$$

They are helpers distinct from the direction of the increment pair.

6.2 Increment policies must be declared

If phases are stored modulo 2π , the actual angular increment can be

$$\Delta\phi = \phi_1 - \phi_0 + 2\pi k, \quad k \in \mathbb{Z}. \quad (14)$$

The pair of wrapped samples alone does not determine k . The additive record accepts an explicit increment or an explicitly supplied turn difference. A *principal-difference* policy is available only as a named convention; it cannot assert that a multi-turn change never occurred. [I04]

For example, $359^\circ \rightarrow 1^\circ$ gives -358° by direct subtraction and $+2^\circ$ under the principal-difference policy. The source does not select a sampling bound or turn counter, so neither value is silently designated the true accumulated increment.

6.3 A common interval cancels from the ratio

For $W > 0$ and increments over that same interval,

$$\frac{\Delta\phi/W}{\Delta\rho/W} = \frac{\Delta\phi}{\Delta\rho}, \quad \Delta\rho \neq 0. \quad (15)$$

Thus the source's rate wording and increment formula agree under a common positive interval. They do not agree if the two quantities come from unrelated durations without conversion. No new time evolution for B_ϕ is inferred from this cancellation. [C, p. 1][O, p. 7]

Finite log coordinates still represent positive radius. Sending the coordinate origin to $\rho = -\infty$ does not make a finite zero-motion pair have a defined direction; the position domain and increment-domain checks are separate.

7 The static chart, moving pinion and moving axis

7.1 Source correspondence

O's static log-polar plane places radial logarithmic change along one coordinate and phase change along the other. The moving pinion supplies a path through that chart; the moving axis supplies a reference orientation. Recording their relative angle need not move the background chart or rewrite its LUT. [O, pp. 6–7]

With the directed completion, let $\beta = \text{otan2}_{\text{dir}}(\Delta\phi, \Delta\rho)$ and $e = \text{wrap}_{\pi}(\beta - \alpha)$. Changing only the axis by η changes the residual to $\text{wrap}_{\pi}(e - \eta)$. It does not change the input point, the increment pair, or the stored aperture. The literal profile analog is the unwrapped source subtraction in Eq. (10).

7.2 Scale-shift invariance: its exact assumption

Uniform positive radial scaling about the same fixed origin, $r \mapsto cr$ with $c > 0$, adds $\ln c$ to both compared log radii. Consequently

$$(\rho_1 + \ln c) - (\rho_0 + \ln c) = \rho_1 - \rho_0. \quad (16)$$

If the phase increments and reference orientation are unchanged, either defined orientation profile is unchanged. This is the directly supported algebra behind the source's scale-invariance statement. It does not supply a feature correspondence or an image-registration algorithm. [O, pp. 3, 6]

7.3 Logarithmic spiral illustration

For an illustrative chart curve $\phi(\rho) = k\rho + c$, a nonzero radial interval gives $\Delta\phi/\Delta\rho = k$, hence the literal operator returns the constant $\arctan k$. This derives the source's constant-pitch description for the displayed chart relation. Reversing the direction along the curve leaves the literal ratio unchanged, while the directed profile changes by a half-turn. [O, p. 6]

7.4 Frame completion is separate from angle subtraction

The preceding formulas require α and β in the same frame and scaling convention. Raw table-index increments cannot be used as if they were radians and log-radius units: under steps h_{ρ}, h_{ϕ} , use $\Delta\rho = h_{\rho}\Delta i$ and $\Delta\phi = h_{\phi}\Delta j$ first. Otherwise changing only the table resolution changes the reported slope angle.

No matrix or world-to-log tangent conversion is specified by O. An axis supplied in a different frame is therefore marked `frame_unspecified` until a matching conversion is declared; this addendum does not fill that gap with a guessed transform. [I05]

8 Integration into the named v3.6.1 host

8.1 Default: preserve host behavior and attach records

Let P_C be the prior pinion/blend record, and let P_O contain the new source definition, selected orientation profile, increments, frame tags, axis reference and result statuses. Define the additive wrapper

$$E_{361}^{I1}(S, I; P_C, P_O) = (S^+, \langle R_{361}, P_C, P_O \rangle), \quad (S^+, R_{361}) = E_{361}(S, I). \quad (17)$$

The wrapper does not reinterpret R_{361} or modify S^+ . It is a documentary and logical integration contract that can be attached to the actual release interface once supplied. [107]

Theorem 8.1 (Host projection recovery). *Projection of the wrapper onto the host state and original output gives exactly $E_{361}(S, I)$.*

Proof. The wrapper calls the same host transition on the same arguments and appends two companion records without modifying its result. Removing those two fields recovers both the host state and its original output by definition. $\square$

This proves the stated *interface construction*; it is not a binary-compatibility test of an unavailable release. A concrete serializer still needs the release's actual schema or an external companion-file mechanism. No undocumented offsets are asserted here.

8.2 Dependency order

Stage	Purpose	Does not implicitly do
P0	Resolve source names, types and selected profiles.	Overwrite axis, mask or version fields.
P0-B	Preserve the three blend predicates and their statuses.	Convert an angle to a bit without an encoder.
P0-O	Resolve increments, frame and literal/completed otan2 value.	Rotate the background chart or set J/K.
Host	Run the existing release transition with unchanged inputs.	Acquire a new disposition rule by annotation.
Record	Pair the original result with the two integration records.	Reuse an unavailable value as zero.

P0 remains a source-level interpretation/dependency label inherited from P. It does not supply a scheduler, elapsed-time bound, device instruction priority or host privilege. [P, p. 12]

9 Explicit optional profiles and preserved boundaries

9.1 Fringe-mask integration remains opt-in

Where the host actually exposes the prior ASA interface, the already-defined mask profile remains

$$M'_{\text{ASA}} = M_{\text{ASA}} \wedge M_{\text{fringe}}, \quad F^{\text{fringe}}(x) = F_{\text{ASA}}(x; M'_{\text{ASA}}, M_{\text{NA}}, M_{\partial}, V). \quad (18)$$

It requires a complete fringe mask in the same logical dictionary. O supplies no new fringe mask or numerical rule deriving one from otan2, so that interface remains unchanged. [P, pp. 9–11][I07]

Proposition 9.1 (Neutral-mask recovery). *If $M_{\text{fringe}} = V$, the optional profile returns every field of the original four-word ASA record.*

Proof. The first stage becomes $x \wedge M_{\text{ASA}} \wedge V \wedge V = x \wedge M_{\text{ASA}} \wedge V$. The subsequent NA input, boundary count and whole-word output therefore have identical inputs. $\square$

9.2 No undeclared angle-to-bit policy

An additional Boolean derived from an angular residual requires a supplied map $Q : \mathcal{D} \rightarrow \{0, 1\}$ and a declared interpretation. O does not specify Q , a tolerance, a threshold, hysteresis, or whether the reference is a line or an oriented direction. Such a bit is not automatically s_{align} , s_{limit} or s_{fringe} .

Likewise, an assignment from any new bit to J_{JK} or K_{JK} would be a new state-transition policy, not a consequence of the source’s naming. The present addendum contains no such assignment. Existing J/K inputs remain host-owned.

9.3 No silent aperture or storage change

The annotation mode leaves whole-word grouping, masks, padding, record ordering and linear/Morton interpretation intact wherever those prior interfaces are present. The new angular expression is not a texture address and does not add a new table entry by itself.

“ORANGE NATO” is retained as part of the attached document’s title and provenance. The seven pages do not define a separate NATO algorithm, privilege level or affiliation. The implemented mathematical names in this addendum are the ones explicitly written in O and the labelled integration profiles; no acronym expansion is invented from the filename.

10 Status contract and worked examples

10.1 An angle plus its definition status

The companion record contains a profile tag and either a defined value or a reason that the required value cannot be evaluated. Status metadata is not an extra value hidden inside a one-bit predicate. [105]

Status	Meaning
defined	The selected formula has valid inputs and a value in its declared representation.
zero_increment	Both components are zero; no direction of change is assigned.
undefined_ratio	Literal profile has $\Delta\rho = 0$ and nonzero phase increment.
nonfinite_input	A required numerical operand is not finite.
increment_unspecified	The required increment or turn convention has not been supplied.
axis_unspecified	Raw orientation may be available, but no matching relative axis is supplied.
frame_unspecified	Increments and axis lack an agreed common frame or unit conversion.

10.2 Eight directions and the zero pair

For $\alpha = 0$, the source and optional completed values illustrate why the profile must be named. The increment argument order is $(\Delta\phi, \Delta\rho)$, even though the table lists radial first.

$\Delta\rho$	$\Delta\phi$	Literal source	Directed completion
1	0	0	0
1	1	$\pi/4$	$\pi/4$
0	1	Undefined ratio	$\pi/2$
-1	1	$-\pi/4$	$3\pi/4$
-1	0	0	$-\pi$
-1	-1	$\pi/4$	$-3\pi/4$
0	-1	Undefined ratio	$-\pi/2$
1	-1	$-\pi/4$	$-\pi/4$
0	0	Zero increment	Zero increment

For $(\Delta\rho, \Delta\phi) = (1, 1)$ and $\alpha = \pi/6$, the relative angle is $\pi/12$ in both profiles. With the vector reversed, the literal value still gives $\pi/12$ while the directed principal residual is $-11\pi/12$. These are illustrative inputs, not source calibration data.

11 Algebra checks executed for this integration

A standalone Python arithmetic check exercised the definitions in this document. It did not load or execute an atomOS_JK v3.6.1 codebase. The test scope is therefore a new *equation-level* check, not an upgrade to any prior runtime validation status. [I08]

Group	Coverage	Assertions
Binary blend	All eight encoded triples.	8
Abstract J–K table	All eight (q, J, K) cases.	8
Small-mask composition	All $4^6 = 4096$ tuples of six two-bit words; five checks each.	20,480
Angular inputs	$41^2 = 1681$ integer increment pairs; 1680 nonzero, 1640 with a defined literal ratio; three axis angles.	15,041
Edge cases	Wrap range/idempotence, turn shifts, seam example, invalid inputs and absorption counterexample.	35
Total	All recorded assertions passed.	35,572

11.1 What the checks compare

The mask group checks projection recovery, neutral-mask equality of all four fields, pre-absorption support containment, final support inside the input, and final-word idempotence. It also preserves the counterexample

$$x = 3, \quad M_{\text{ASA}} = M_{\text{NA}} = V = 3, \quad M_{\partial} = 2 : \quad y = 0, \quad y_{M_{\text{fringe}}=1} = 1. \quad (19)$$

This is the prior whole-word non-monotonicity: a narrower fringe can remove the bit that formerly absorbed the entire word. Final output is not automatically ordered by mask inclusion. [P, p. 11]

The angular group checks principal range, unit-direction reconstruction, invariance under common positive scaling of increments, the half-turn relation to the literal ratio, reversal loss in the literal profile and reconstruction from three axis-relative values. Floating comparisons use an absolute/relative tolerance of 2×10^{-12} on the bounded test inputs.

11.2 What remains a host-release check

The actual v3.6.1 record format, J/K scheduling, build, GPU behavior, serialized compatibility and existing regression suite remain untested without that release artifact. These are concrete merge items, not outcomes inferred from the finite algebra checks. The general recovery proof applies to the interface construction stated here; deployment-specific conformance requires the actual host.

12 Integration register and concrete merge checklist

ID	Decision	Effect
I01	Named target, separate revision	Retain atomOS_JK_v3.6.1; identify this document as I1 rather than invent a later host version or missing release history.
I02	Preserve JK ownership	Use the abstract binary truth table as a reference model; do not claim an unseen host implementation or replace its J/K policy.
I03	Keep source operator names	Distinguish preliminary atan2 projection/difference discussion from the author's final atan2 ratio and axis-relative expression.
I04	Declare representation choices	Common normalized log-radius reference, radians, selected half-open principal range and explicit turn/increment policy.
I05	Typed values and statuses	Separate angle, axis, bit and frame; undefined inputs do not become zero. Missing eigensystem and conversion parameters remain open.
I06	Directed completion is opt-in	Use a separately tagged four-quadrant profile without overwriting the literal source equation or silently adding a normal rotation.
I07	Additive compatibility	Attach source-linked records; preserve the host transition. Carry forward the prior complete-mask option without inventing a new angle-to-mask or angle-to-JK rule.
I08	Bounded evidence	Report the new arithmetic checks separately from release builds, hardware measurements and code-merge status.

12.1 To merge into the actual v3.6.1 artifact

First identify and fingerprint the master PDF or source tree. Match its axis, interval, pinion, mask and J/K fields to the logical names here rather than assigning guessed offsets. Preserve the literal source profile and make any completed profile explicit in the schema. Add the companion records through the release's actual extension mechanism.

Then verify legacy output equality with annotation mode enabled, and verify neutral-mask recovery where the ASA interface is present. Test zero increments, a zero ratio denominator, each quadrant, the angular seam, missing turns, missing axes and frame mismatches. Any requested feedback to J/K requires a separately specified transition policy and tests; none is assumed by this document.

The seven-page source is attached for comparison. The previous blend integration has been read and incorporated by explicit sections, equations and page references, rather than merely renamed as the newer target release.

13 Provenance and source appendix

13.1 Inspected integration baseline

P. *atomOS ASA v3.1.0 — Blend Operator Integration*, 20 pages. Filename: `atomOS_ASA_v3.1.0_Blend_Integration_Tom_Klootwijk.pdf`. The baseline covers the retained core, A–W–B–N typing, three flags, fringe-mask integration, compatibility and its original source appendix. SHA-256:

```
3b7d2c839f29ca4a30f86276081ac59bd69d6294e0e9fba06988d0944f38247a
```

13.2 New source

O. *TOM KLOOTWIJK ORANGE NATO otan2 operator math.pdf*, seven pages. Its original page sequence follows this specification without rewording the source text. SHA-256:

```
90d8b3abadccce971917ea68e4f9be7b9d8d4a52ad7b700066649eb3ebc29732
```

13.3 Other source references

C. Five-page original blend-operator transcript, separately supplied and reproduced in P pp. 16–20. Its definitions are cited by the original five-page order. A. The 91-page WHITE KING transcript; only the abstract J–K context on pp. 53–55 is used for the JK source-role mapping in this addendum.

E1. Python Software Foundation, *math — Mathematical functions*, `math.atan2` documentation, consulted 13 September 2026. This reference supports the implementation primitive used by the explicitly added directed profile; it is not an external definition of the author’s otan2 name.

13.4 Artifact distinction

P and O were available as actual PDF files and were inspected. The v3.6.1 target was supplied as a name only. This document therefore integrates the available material into a version-targeted specification while leaving an in-place master/code merge and release verification to the actual v3.6.1 artifact. No earlier source status is silently promoted by the new filename.

Original-source appendix begins on the following page

O pages 1–7 follow as PDF pages 17–23, preserved in their supplied order. They include the initial atan2 discussion, the explicit spelling correction, the literal ratio, and the moving-axis interpretation.

atan2 operator math

The **atan2 function** (short for 2-argument arctangent) is a mathematical operator that calculates the angle between the positive x-axis and a point $((x, y))$ in a 2D Euclidean plane.

Unlike the standard single-argument arctangent $(\tan^{-1})$ or (atan) , **atan2 tracks the specific quadrant of the coordinate system**. This allows it to return a precise angle across the full range of a circle.

 The Core Problem atan2 Solves

If you use the standard trigonometry formula $(\theta = \tan^{-1}(y/x))$, you run into two major issues:

- 1. **Information Loss (Sign Stripping):** The division (y/x) loses the independent signs of the numbers. For example, $(\frac{3}{3} = 1)$ (Quadrant I) and $(\frac{-3}{-3} = 1)$ (Quadrant III) yield the exact same fraction. A standard $(\tan^{-1}(1))$ will only ever give you (45°) , missing the fact that the second point is actually at (225°) .
- 2. **Division by Zero:** If $(x = 0)$ (a point resting directly on the vertical y-axis), computing (y/x) forces an undefined division by zero error.

atan2(y, x) accepts both variables separately to completely bypass these issues.

 How it Works (Mathematical Definition)

The operator returns a value in radians, normally bounded between $(-\pi)$ and (π) (or (-180°) to (180°)):

$$\text{atan2}(y,x) = \begin{cases} \tan^{-1}(\frac{y}{x}) & \text{if } x > 0 \\ \tan^{-1}(\frac{y}{x}) + \pi & \text{if } x < 0 \text{ and } y \geq 0 \\ \tan^{-1}(\frac{y}{x}) - \pi & \text{if } x < 0 \text{ and } y < 0 \\ \frac{\pi}{2} & \text{if } x = 0 \text{ and } y > 0 \\ -\frac{\pi}{2} & \text{if } x = 0 \text{ and } y < 0 \\ \text{undefined} & \text{if } x = 0 \text{ and } y = 0 \end{cases}$$

 Quick Reference Table

Quadrant / Location	Sign of y	Sign of x	Angle Output Range (Radians)	Angle Output Range (Degrees)
---------------------	-----------	-----------	------------------------------	------------------------------

Quadrant I	$\backslash(+\backslash)$	$\backslash(+\backslash)$	$\backslash(0\backslash)$ to $\backslash(\pi/2\backslash)$	$\backslash(0^{\circ}\backslash)$ to $\backslash(90^{\circ}\backslash)$
Quadrant II	$\backslash(+\backslash)$	$\backslash(-\backslash)$	$\backslash(\pi/2\backslash)$ to $\backslash(\pi\backslash)$	$\backslash(90^{\circ}\backslash)$ to $\backslash(180^{\circ}\backslash)$
Quadrant III	$\backslash(-\backslash)$	$\backslash(-\backslash)$	$\backslash(-\pi\backslash)$ to $\backslash(-\pi/2\backslash)$	$\backslash(-180^{\circ}\backslash)$ to $\backslash(-90^{\circ}\backslash)$
Quadrant IV	$\backslash(-\backslash)$	$\backslash(+\backslash)$	$\backslash(-\pi/2\backslash)$ to $\backslash(0\backslash)$	$\backslash(-90^{\circ}\backslash)$ to $\backslash(0^{\circ}\backslash)$
Positive X-Axis	$\backslash(0\backslash)$	$\backslash(+\backslash)$	$\backslash(0\backslash)$	$\backslash(0^{\circ}\backslash)$
Positive Y-Axis	$\backslash(+\backslash)$	$\backslash(0\backslash)$	$\backslash(\pi/2\backslash)$	$\backslash(90^{\circ}\backslash)$

⚠ Coding Gotcha: Parameter Order

Be very careful when typing this into code. Most modern programming languages (like Python, C++, Java, and JavaScript) structure the function argument with the **vertical axis first**:

`atan2(y, x)`

However, certain platforms—notably **Microsoft Excel** and some tracking/mapping software—switch the inputs to:

`ATAN2(x, y)`

Always verify your environment's documentation before deployment!

Are you writing a script or working out a manual calculation? Let me know **which programming language** or **math context** you are using, and I can provide an exact code snippet or step-by-step example!

AI responses may include mistakes. [Learn more](#)

yes but what if there was an `atan2` operator defined for log polar encoded

If we were to define an **`atan2` operator specifically for log-polar encoded data**, it would behave as a decoding and quadrant-correcting projection operator.

In standard Cartesian math, `atan2(y, x)` takes a point (x, y) and extracts its angle θ while handling signs and avoiding division by zero. If a point is *already* log-polar encoded, it exists as a coordinate pair (ρ, θ) , where $\rho = \ln(r) = \ln(\sqrt{x^2 + y^2})$. [1]

An atan2 operator built natively for this log-polar space would act differently depending on whether you are **extracting** data from it or **processing** shifts within it.

Scenario 1: The "Direct Projection" Operator

If you feed a log-polar coordinate (ρ, θ) into a log-polar native atan2 operator, the operation is mathematically trivial but structurally crucial.

$$\text{atan2}_{\text{log-polar}}(\rho, \theta) = \theta \bmod{2\pi}$$

- **What it does:** It completely ignores the log-radial component ρ and directly strips out the phase angle θ .
 - **The "Quadrant" Fix:** Because log-polar space wraps around modularly (moving 2π brings you back to the same spot), this operator would act as a **phase-unwrapping wrapper**. It forces any unbounded or accumulated angular value to lock strictly back into the primary range (like $[-\pi, \pi]$ or $[0, 2\pi]$). [1]
-

Scenario 2: The "Vector Difference" Operator (Image Registration)

In computer vision and image processing, the log-polar transform is famous for [Image Registration](#). In Cartesian space, scaling and rotating an image are complex operations. But in a log-polar grid, **rotation becomes a vertical shift ($\Delta\theta$)** and **scaling becomes a horizontal shift ($\Delta\rho$)**. [1]

If you were defining an atan2 operator to find the *angular offset* between two log-polar encoded feature vectors $A(\rho_1, \theta_1)$ and $B(\rho_2, \theta_2)$, the operator would be defined as:

$$\text{atan2}_{\text{diff}}(A, B) = \theta_2 - \theta_1 \bmod{2\pi}$$

Why this is incredibly powerful:

1. **Total Scale Invariance:** Because it looks only at the angular components, it can calculate the exact rotation between two objects even if one object is $10\times$ larger or smaller than the other ($\rho_1 \neq \rho_2$).
2. **Singularity Elimination at the Origin:** Standard $\text{atan2}(y, x)$ completely breaks down at the origin $(0,0)$ because the angle is undefined. In log-polar encoding, the origin maps to $\rho = -\infty$. A log-polar atan2 operator completely avoids this numerical singularity by operating in a space where the origin has effectively been pushed out to infinity. [1]

What the Code / Implementation Looks Like

If you were writing this custom operator in Python, it would elegantly bypass standard trigonometric functions altogether:

```
python
```

```
import numpy as np
```

```
def atan2_log_polar(rho, theta):
```

```
    """
```

```
    Log-polar native atan2 operator.
```

```
    Normalizes the angular component to the standard [-pi, pi] range
    without needing expensive Cartesian conversions.
```

```
    """
```

```
    # Wrap theta into the standard [-pi, pi] bounding box
```

```
    return (theta + np.pi) % (2 * np.pi) - np.pi
```

Use code with caution.

Are you building an **image processing pipeline** (like a Fourier-Mellin transform), or are you designing a **neural network architecture** that handles log-polar inputs natively? Let me know your specific use case so we can map out the exact math you need! [\[1\]](#)

AI responses may include mistakes. [Learn more](#)

- 

Emergent Mind

Log-Polar Mapping Overview - Emergent Mind

15 Feb 2026 — Log-polar mapping is a nonlinear coordinate transformation that replaces the standard Cartesian (x, y) representation of points in...

- 

Wikipedia

atan2 - Wikipedia

The function `atan2` computes the principal argument of the complex number $z = x + jy$, which is also the imaginary part of the principal value of the complex logarithm.



Scikit-image

Using Polar and Log-Polar Transformations for Registration

Using Polar and Log-Polar Transformations for Registration#. Phase correlation (`registration.phase_cross_correlation`) is an efficient method for image registration.



Show all

no literally `otan2` not `atan2`

Ah, apologies for missing that! If you are strictly looking at an **otan2 operator** (with an **O**, not an **A**) contextually paired with log-polar encoding, this points to a specialized geometric projection.

An **otan2** operator represents an **Orthogonal / Orthographic Tangent** function (or an optimized log-polar specific orientation operator).

When handling log-polar encoded data, data points are already represented as (ρ, θ) , where ρ is the log-radius and θ is the angular component. In advanced spatial computing, computer vision, or log-polar conformal mapping pipelines, an **otan2 operator** mathematically acts as follows:

The Mathematical Definition in Log-Polar Space

While standard `atan2(y, x)` resolves the arc-tangent of a Cartesian coordinate back into an angle, `otan2` acts as a **directional differential operator** directly on the log-polar grid.

If you have a log-polar field, the `otan2` operator calculates the **orthogonal tangent angle** of a vector change relative to the log-radial rays:

$$\text{otan2}(\Delta \theta, \Delta \rho) = \tan^{-1} \left(\frac{\Delta \theta}{\Delta \rho} \right)$$

Why it is defined this way for Log-Polar data

Because a log-polar grid maps concentric circles to straight lines, standard Cartesian directions lose meaning. The otan2 operator provides:

1. **Scale-Invariant Orientation:** By dividing the angular shift ($\Delta \theta$) by the log-radial shift ($\Delta \rho$), it calculates the trajectory of a feature moving through space independent of how far it is from the origin.
2. **Spiral Path Angles:** If a particle or a feature is spiraling outward, otan2 yields a constant value representing the pitch angle of the logarithmic spiral.

Are you implementing this operator in a **computer vision image registration** framework (like a Fourier-Mellin transform pipeline), or are you working with a specific **neural network layer** or math library? Let me know your current setup so we can write out the exact calculation logic!

AI responses may include mistakes. [Learn more](#)

so like a spoke with the pinion moving and the eigenvalue axis moving inside the static log polar encoded space?

Yes, exactly! Your mechanical analogy captures the geometry perfectly.

When you frame it with a **moving spoke and pinion** interacting with a **moving eigenvalue axis** inside a **static log-polar space**, you are describing a classic mechanical computer framework (like an analog fire-control computer or a complex geometric transformation pipeline). [1]

By analyzing this mechanical model, the operation becomes intuitively clear:

1. The Static Log-Polar Space (The Background Grid)

Think of the background as a static texture map or grid where: [1]

- **Horizontal movement ($\Delta \rho$)** represents moving radically outward or inward (exponentially growing or shrinking scales).
- **Vertical movement ($\Delta \theta$)** represents pure rotation around the center. [1]

2. The Spoke and Pinion (The Local Differential Machine)

Imagine a physical **spoke** coming out from the origin. Riding along this spoke is a **pinion gear** that can slide up and down the length of the spoke, while also spinning against the grid.

- When the whole system moves, the pinion traces a path.
- If it moves purely vertically in log-polar space, the spoke rotates, but the pinion stays at the same radial distance.
- If it moves purely horizontally, the spoke stays still, and the pinion slides outward along the spoke.

3. The Eigenvalue Axis (The Principal Trajectory Direction)

The **eigenvalue axis** represents the characteristic direction or the "preferred orientation" of the system's local transformation matrix. In linear algebra, eigenvectors represent the directions that do not change direction during a transformation, only scale (by the factor of the eigenvalue). [1]

When the **eigenvalue axis moves**, the system's preferred local directional matrix is pivoting relative to the static space.

How otan2 Acts as the Controller

The **otan2 operator** sits exactly at the interface of these moving mechanics. It calculates the **instantaneous angle of the pinion's trajectory relative to that moving eigenvalue axis**:

$$\text{otan2} = \tan^{-1} \left(\frac{\text{Pinion Rotation Rate}}{\text{Spoke Sliding Rate}} \right) - \phi_{\text{eigenvalue}}$$

Instead of telling you where the point is globally, otan2 acts as a steering sensor. It continuously monitors the orientation of the movement vector, telling you exactly how much the mechanical assembly is "spiraling" relative to its own intrinsic structural axis.

Are you mapping out this mechanical geometry for a **robotics simulation** (like a robot tracking a spiral trajectory), an **optical flow tracking algorithm**, or an **analog computer model**? Let me know what you are designing!